

Grundlagen der Algorithmen

Zusammenfassung und Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Max Mustermann
Matrikelnummer 12345678
Studiengang Informatik B.Sc.
Semester Sommersemester 2025
Datum 3. Mai 2026

Inhaltsverzeichnis

1	Grundlagen der Algorithmen	2
1.1	Definition	2
1.2	Eigenschaften eines korrekten Algorithmus	2
1.3	Vorteile und Nachteile formaler Algorithmen	2
1.4	Wichtiger Hinweis zur Komplexitätsanalyse	3
2	Laufzeitkomplexität	3
2.1	Überblick und Notation	3
2.2	Abbildung: Wachstum der O-Klassen	3
2.3	Vergleich bekannter Sortieralgorithmen	3
2.3.1	Zwei Tabellen nebeneinander (<code>\sidetables</code>)	4
3	Implementierungsbeispiel	4
3.1	Merge Sort in Python	4
3.2	Laufzeit-Analyse des Merge-Sort-Schritts	4

1 Informations- und Hinweis-Boxen

1.1 infobox — Infos und Definitionen

Definition: Algorithmus

Ein **Algorithmus** ist eine endliche, eindeutige Folge von Anweisungen, die ein Problem in endlicher Zeit löst. Jeder Algorithmus muss *terminieren*, *korrekt* und *eindeutig* sein.

Infoboxen können auch ohne Titel verwendet werden — z. B. für kurze Hinweise oder ergänzende Informationen im Fließtext.

1.2 warnbox — Warnhinweise und Fallstricke

Off-by-one-Fehler

Beim Zugriff auf Array-Indizes stets prüfen, ob der Index im Bereich $[0, n - 1]$ liegt. Ein häufiger Fehler ist der Zugriff auf Index n , was zu einem `IndexOutOfBoundsException` führt.

1.3 merksatz — Wichtige Sätze und Regeln

Master-Theorem

Für Rekurrenzen der Form $T(n) = a \cdot T(n/b) + f(n)$ gilt:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{falls } f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{falls } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{falls } f(n) = \Omega(n^{\log_b a + \varepsilon}) \end{cases}$$

Merksätze können auch ohne Titel stehen — etwa für kurze, prägnante Regeln die man unbedingt im Kopf behalten muss.

1.4 beispiel — Konkrete Beispiele

Beispiel: Quicksort auf $[3, 1, 4, 1, 5, 9]$

Pivot = 3 (erstes Element). Partition ergibt:

- Links ≤ 3 : $[1, 1]$
- Rechts > 3 : $[4, 5, 9]$

Rekursiver Aufruf auf beide Teile, dann zusammensetzen: `[1, 1, 3, 4, 5, 9]`.

1.5 aufgabe — Übungsaufgaben

Aufgabe 1: Laufzeitanalyse

Bestimmen Sie die durchschnittliche Zeitkomplexität von Quicksort mithilfe des Master-Theorems. Welche Rekurrenz beschreibt den Algorithmus?

Aufgabe 2

Implementieren Sie Mergesort in Python und vergleichen Sie die Laufzeit empirisch mit Bubblesort für $n \in \{100, 1000, 10000\}$.

2 Strukturelemente

2.1 process — Schrittfolgen

- 1 Pivot-Element aus der Liste wählen (z. B. erstes, letztes oder mittleres)
- 2 Liste in zwei Teile partitionieren: Elemente $\leq$ Pivot links, $>$ Pivot rechts
- 3 Quicksort rekursiv auf den linken Teilbereich anwenden
- 4 Quicksort rekursiv auf den rechten Teilbereich anwenden
- 5 Ergebnis: links + [pivot] + rechts

2.2 procon — Vor- und Nachteile

Vorteile

- Sehr schnell in der Praxis ($O(n \log n)$ im Durchschnitt)
- In-place — kein zusätzlicher Speicher nötig
- Cache-freundlich durch sequentiellen Speicherzugriff

Nachteile

- Worst-case $O(n^2)$ bei schlechter Pivot-Wahl
- Nicht stabil — gleiche Elemente können die Reihenfolge tauschen
- Rekursionstiefe kann bei großen n zum Stack-Overflow führen

2.3 konzeptkarte — Strukturierte Übersichtskarte

Die konzeptkarte-Umgebung fasst Beschreibung, Tabelle und Ablauf in einer visuellen Karte zusammen. Mit `\kzweispaltig` lassen sich zwei Inhalte nebeneinander setzen.

Mergesort

Ein **stabiler**, rekursiver Sortieralgorithmus nach dem Teile-und-Herrsche-Prinzip. Zerlegt die Liste in zwei Hälften, sortiert diese rekursiv und *mischt* sie dann zusammen.

- 1 Liste halbieren
- 2 Linke Hälfte rekursiv sortieren
- 3 Rechte Hälfte rekursiv sortieren
- 4 Beide Hälften zusammenmischen

Eigenschaft	Wert
Zeitkomplexität	$O(n \log n)$ immer
Speicher	$O(n)$ extra
Stabil	Ja
In-place	Nein

Heapsort

Nutzt einen **Max-Heap**, um das größte Element zu finden und an die richtige Position zu setzen. Nicht stabil, aber in-place und garantiert $O(n \log n)$.

- 1 Max-Heap aufbauen
(`heapify`)
- 2 Wurzel (Maximum) ans Ende tauschen
- 3 Heap-Größe um 1 reduzieren
- 4 Schritte 2–3 wiederholen

Eigenschaft	Wert
Zeitkomplexität	$O(n \log n)$ immer
Speicher	$O(1)$ extra
Stabil	Nein
In-place	Ja

3 Code-Elemente

3.1 codebox — Einfacher Monospace-Block

Für kurze Code-Fragmente ohne Syntax-Highlighting: `codebox`.

```
Eingabe: [5, 2, 9, 1, 7]
Nach Pass 1: [2, 5, 1, 7, 9]
Nach Pass 2: [2, 1, 5, 7, 9]
Nach Pass 3: [1, 2, 5, 7, 9]
Fertig nach 4 Durchläufen.
```

3.2 syntaxbox — Code mit Syntax-Highlighting

3.2.1 Python

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + mid + quicksort(right)

# Beispielaufruf
data = [3, 6, 8, 10, 1, 2, 1]
print(quicksort(data)) # [1, 1, 2, 3, 6, 8, 10]
```

3.2.2 Java

```
public class Mergesort {

    public static int[] sort(int[] arr) {
        if (arr.length <= 1) return arr;
        int mid = arr.length / 2;
        int[] left = sort(Arrays.copyOfRange(arr, 0, mid));
        int[] right = sort(Arrays.copyOfRange(arr, mid, arr.length));
        return merge(left, right);
    }

    private static int[] merge(int[] a, int[] b) {
        int[] result = new int[a.length + b.length];
        int i = 0, j = 0, k = 0;
        while (i < a.length && j < b.length)
            result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
        while (i < a.length) result[k++] = a[i++];
        while (j < b.length) result[k++] = b[j++];
        return result;
    }
}
```

3.2.3 SQL

```
-- Ergebnisse nach Score sortiert (analog zu ORDER BY = Sortieralgorithmus)

SELECT student_id, name, score
FROM exam_results
WHERE score >= 50
ORDER BY score DESC, name ASC
LIMIT 10;
```

3.3 \icode — Inline-Code im Fließtext

Der Befehl `\icode{...}` hebt Code im Fließtext hervor: Die Funktion `mergesort(arr)` hat eine Zeitkomplexität von $O(n \log n)$. Die Variable `pivot` speichert das Pivot-Element. Fehler vom Typ `NullPointerException` treten auf, wenn `arr == null`.

4 Tabellen und Layout

4.1 stdtable — Formatierte Tabellen

Algorithmus	Strategie	$\emptyset$	Worst-case
Quicksort	Teile-und-Herrsche	$O(n \log n)$	$O(n^2)$
Mergesort	Teile-und-Herrsche	$O(n \log n)$	$O(n \log n)$
Heapsort	Heap	$O(n \log n)$	$O(n \log n)$
Bubblesort	Vergleich	$O(n^2)$	$O(n^2)$
Timsort	Hybrid	$O(n \log n)$	$O(n \log n)$

Spaltendefinition

`l`, `r`, `c` sind fest breite Spalten. `x` füllt den verbleibenden Platz automatisch.
`p{3cm}` ist eine manuell festgelegte Breite.

4.2 sidetables — Zwei Inhalte nebeneinander

Struktur	Zugriff
Array	$O(1)$
Linked List	$O(n)$
Hash Map	$O(1)$ avg.
BST	$O(\log n)$

Struktur	Einfügen
Array	$O(n)$
Linked List	$O(1)$
Hash Map	$O(1)$ avg.
BST	$O(\log n)$

5 Konfiguration der Vorlage

Anpassung in drei Dateien

1. **config/colormeta.tex** — Farben, Schriften, Abstände (*hier alles anpassen*)
2. **config/usermeta.tex** — Name, Studiengang, Matrikelnummer
3. **config/topicmeta.tex** — Titel, Fach, Semester, Beschreibung

Goldene Regel

Farben und Schriften werden *ausschließlich* in `colormeta.tex` definiert. Nirgendwo sonst im Dokument werden Hex-Werte oder Schriftnamen hart kodiert — alle Umgebungen referenzieren nur Farbnamen wie `primary`, `accent`, `infoframe` usw.

5.1 Schriften anpassen

```
% Standard (immer in TeX Live verfügbar):  
\newcommand{\mainfont}{TeX Gyre Pagella}  
\newcommand{\sansfont}{TeX Gyre Heros}  
\newcommand{\monofont}{TeX Gyre Cursor}  
  
% Premium-Alternativen (Homebrew: brew install --cask font-source-serif-  
4):  
\newcommand{\mainfont}{Source Serif 4}  
\newcommand{\sansfont}{Source Sans 3}  
\newcommand{\monofont}{JetBrains Mono}
```

Aufgabe 3: Template anpassen

Ändern Sie die Farben `primary` und `accent` in `colormeta.tex` auf eigene Werte und kompilieren Sie mit `make build`. Beobachten Sie, wie sich alle Boxen, Tabellen und Überschriften automatisch anpassen.